

CubeDynamics: An Inspectable Grammar for Environmental Data Analysis

Ty Tuff

Environmental Data Science Innovation & Impact Lab
Cooperative Institute for Research in Environmental Sciences
University of Colorado Boulder, Boulder, Colorado, USA
[ORCID 0000-0001-5249-5197](https://orcid.org/0000-0001-5249-5197)

September 25, 2026

Abstract

Environmental questions that sound simple often compress consequential choices about observations, transformations, alignment, and assumptions into short scripts. Making a script rerunnable supports computational repeatability, but does not by itself reveal what scientific question the code asked. CUBEDYNAMICS grew from a recurring need across environmental projects: retain a readable analytical sequence without replacing xarray’s labeled multidimensional data structures or hiding the choices underneath them. The open-source Python package provides named environmental inputs that retain provider and product identity, named transformations with explicit parameters, execution in the order written, a running description of the package’s interpretation of the current result, and a record of how it was produced. Its intended users are environmental researchers and research software engineers who want reusable analyses that collaborators can inspect and extend. Implementation and semantic validation of the public 0.1.0rc3 candidate and a dated post-rc3 development record show that Boolean condition outputs and summaries are described consistently with their numerical contents, meaning-changing operation orders remain distinct, mismatches detectable from coordinates and observation-support metadata are reported, and known contract-incompatible sequences are rejected with explanations. A South Dakota vignette demonstrates a limited two-input combination of temperature and precipitation conditions. The objective is not to make difficult environmental synthesis appear simple, but to make its complexity readable while keeping the observations, transformations, and evidence underneath recoverable.

1 Introduction

The question *Where was it hot and dry?* sounds modest. An analyst could load temperature and precipitation, define two conditions, combine them, and make a map. Yet each step opens another question: Which products? Which dates and grids, and what spatial area or time interval does each value represent? Is “hot” an absolute threshold, a local quantile, or an anomaly from a climatology? What counts as “dry”? How are missing observations handled? Were the sources regridded or matched in time, and in what order were values transformed and summarized? A short question can require substantial computational machinery.

There is a deeper problem. Even perfectly rerunnable code may not make the question inspectable. Consider the same 2×2 temperature field, threshold, and spatial mean in Figure 1. Thresholding first asks what fraction of cells satisfy a condition. Averaging first asks whether the regional mean satisfies it. Both sequences can be computationally reproducible; they are different scientific statements.

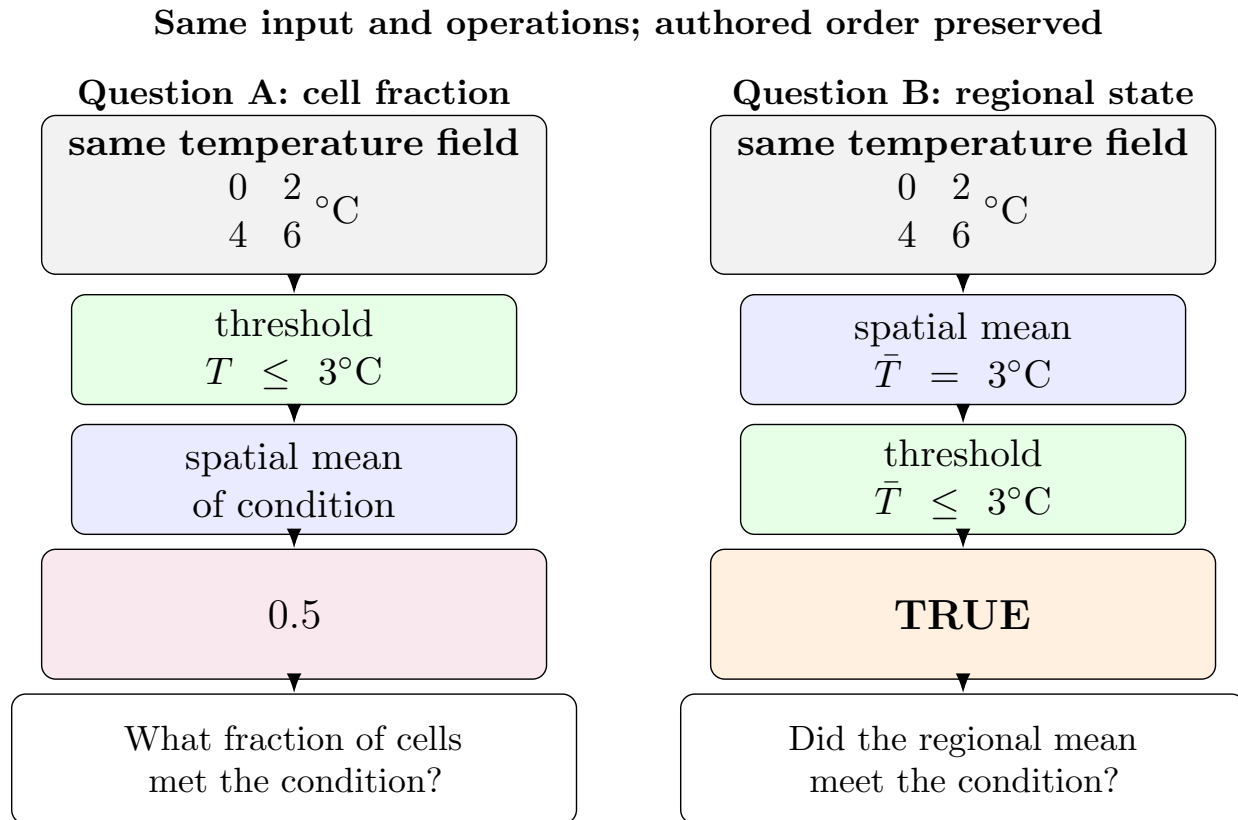


Figure 1: Authored order changes scientific meaning. For the same field $\{0, 2, 4, 6\}^\circ\text{C}$ and inclusive threshold $T \leq 3^\circ\text{C}$, threshold-then-mean produces a cell fraction of 0.5, whereas mean-then-threshold produces one regional Boolean state, **TRUE**. CUBEDYNAMICS executes both statements and preserves rather than equates them.

Computational reproducibility has several meanings, and successful re-execution is only one component of transparent research (Peng, 2011; Goodman et al., 2016; National Academies of Sciences, Engineering, and Medicine, 2019; Essawy et al., 2020). Here *scientific inspectability* denotes a narrower property: a reviewer can recover the provider and product, exact data request, parameters, order of operations, changes in what the intermediate results represent, record of completed steps, and evidence limited to what was actually tested. Inspectability does not establish that the statement is scientifically appropriate or fit for a decision. It makes the choices available for examination. Throughout this paper, *observation* is used broadly for a value supplied by an environmental data product; depending on the source, that value may be a direct measurement, an interpolated grid, a remotely retrieved quantity, a modeled estimate, or a derived variable. The source and procedure therefore remain part of its meaning (Madin et al., 2007; Janowicz et al., 2019; Haller et al., 2019).

CUBEDYNAMICS is intended first for environmental researchers and research software engineers who already use xarray’s labeled multidimensional arrays, computational notebooks, and geospatial

libraries, but whose reusable scripts must cross products, projects, and collaborators. The package adds shared names for environmental inputs and transformations and, after each step, records what the current result represents and how it was produced. It does not introduce another numerical data structure or a system for scheduling jobs. Its central question is:

What exactly did this analysis ask the observations?

This paper explains where that design came from, how its readable expression connects to an inspectable foundation, how new sources and project methods can extend the vocabulary, and what the public 0.1.0rc3 candidate and explicitly separated later evidence demonstrate. The accompanying Supplementary Information (SI) contains the complete input and output rules, source inventories, validation records, and specialized extensions.

2 From narrative pipelines to a shared grammar

My route to CUBEDYNAMICS began with ways of making analysis readable. Fry’s computational information-design pipeline—acquire, parse, filter, mine, represent, refine, and interact—presented data work as a directional process whose stages contributed to an interpretable whole (Fry, 2004, 2007). I later encountered ggplot2, an R visualization package built as a layered implementation of Wilkinson’s Grammar of Graphics (Wilkinson, 2005; Wickham, 2010). In that grammar, a plot is specified by composing data, visual mappings, geometric marks, statistical transformations, scales, and coordinates, making much of the visual argument visible in the code. The tidyverse broadened that experience beyond plotting. It is a collection of R packages that share design conventions for importing, reshaping, transforming, visualizing, and modeling primarily tabular data (Wickham, 2014; Wickham et al., 2019). Within that ecosystem, named operations can be composed in the order an analyst reasons through them.

When my work moved from R data frames to larger environmental arrays in Python, xarray provided arrays whose dimensions and coordinates have names (Hoyer and Hamman, 2017), while Dask allowed those arrays to be evaluated in parallel or only when needed (Rocklin, 2015). I missed the narrative continuity of the earlier R workflow. The analysis still had a sequence, yet its data sources and steps that changed a field into a condition or summary were easily dispersed across loaders, helper functions, array expressions, and metadata. CUBEDYNAMICS began as an attempt to recover that readable progression while addressing a different problem: environmental data need their source and spatial and temporal meaning retained as they are transformed, together with evidence that generic data-manipulation verbs do not supply by themselves.

This lineage should not be read as a feature-by-feature comparison between the tidyverse and CUBEDYNAMICS. The tidyverse is an R ecosystem centered largely on tabular analysis; CUBEDYNAMICS is a Python package designed to wrap multidimensional environmental objects already handled by xarray and related libraries. A researcher would not normally choose between them as interchangeable implementations. The connection is conceptual: both use a small vocabulary of named operations and written order to make analytical reasoning easier to follow.

That route is personal, but the design problem is shared. Information-visualization research has long represented analysis as transformations among data states (Chi, 2000). Literate programming argued

that programs should be organized for human explanation as well as machine execution (Knuth, 1984). Computational notebooks place executable work beside narrative but expose a recurring tension between exploration and explanation (Rule et al., 2018). More general workflow scholarship treats analysis as a progression of consequential choices rather than a single calculation (Stoudt et al., 2021). CUBEDYNAMICS belongs to this broader lineage. Its particular focus is what environmental data and intermediate results mean as they move through an executable analysis: what was observed, from which product, over what spatial area or time interval, through which transformations, and with what evidence.

The idea then grew through workflows involving climate and vegetation observations, fire objects, terrain, roads, streamflow, synchrony methods, teaching notebooks, and decision vignettes. These projects differed scientifically, but repeatedly needed the same connective layer: a recognizable name for the environmental information entering an analysis, explicit transformations written in analytical order, and a way to inspect what object existed after each stage. The need was not another storage system. It was a shared interface between project methods and the numerical infrastructure beneath them.

That infrastructure is already mature. NumPy supplies general numerical arrays (Harris et al., 2020); xarray adds named dimensions, coordinates, and Dataset structures (Hoyer and Hamman, 2017); and Dask supports delayed and parallel computation (Rocklin, 2015). Rasterio and rioxarray read and write gridded raster data (Gillies and contributors, 2013–present; Snow et al., 2026), while GeoPandas and Shapely represent vector features and geometric operations (Jordahl et al., 2020; Gillies et al., 2025). The STAC standard describes discoverable spatiotemporal assets (SpatioTemporal Asset Catalog community, 2024). Data-cube systems organize large Earth-observation collections for multidimensional analysis (Giuliani et al., 2017; Lu et al., 2018; Appel and Pebesma, 2019).

An Earth system data cube is a general organizational concept rather than a single package: diverse variables are arranged as labeled multidimensional data, often on a shared spatial and temporal grid, so they can be accessed and analyzed together (Mahecha et al., 2020; Montero et al., 2024). A recent survey in *Environmental Data Science* describes how such cubes reduce technical preparation while emphasizing that their life cycle still presents scientific challenges involving interpolation, aggregation, distortion, autocorrelation, metadata, and domain-specific interpretation (Montero et al., 2024). Related work develops analysis-ready satellite products (Dwyer et al., 2018), interoperability among cube systems (Giuliani et al., 2019), concept-aware Earth-observation cubes (Augustin et al., 2019), and language-independent process graphs for executing analyses on remote services (Schramm et al., 2021). CUBEDYNAMICS does not build or store a data cube. It works with the resulting ordinary Python objects and concentrates on making the analytical statement and its changing scientific meaning inspectable.

Adjacent systems make other parts of analytical structure explicit, although they are not direct substitutes for one another. ArcGIS ModelBuilder is a visual programming environment inside desktop GIS: users connect spatial data and geoprocessing tools in a diagram, with one tool’s output becoming another’s input (Esri, 2026). GeoJModelBuilder provides a related open-source approach to graphical geoprocessing workflows (Zhang et al., 2017). Pegasus is a system for planning and running computational workflows (Deelman et al., 2015). The Common Workflow Language is a standard for describing portable command-line workflows across execution environments (Crusoe et al., 2022). WINGS adds domain knowledge and constraints to help users construct computational experiments whose components are compatible (Gil et al., 2011). CUBEDYNAMICS instead operates within an ordinary Python analysis. These systems are cited as precedents for visible sequence,

portable execution, or scientifically constrained composition, not as products an environmental Python user would necessarily compare side by side.

Environmental Python users are more likely to encounter CUBEDYNAMICS alongside domain packages than as a replacement for them. `xclim` supplies tested climate indicators on `xarray` objects (Bourgault et al., 2023); `clisops` performs climate-data subsetting and related operations (clisops contributors, 2026); `intake-esm` catalogs and loads Earth-system model collections (intake-esm contributors, 2026); and `ESMValTool` organizes established model-evaluation recipes (Righi et al., 2020). CUBEDYNAMICS does not reproduce those numerical or domain libraries. Its proposed contribution is the combination of source-qualified entry points, runtime descriptions of intermediate scientific objects, authored-order rules, a local operation trace, ordinary `xarray` interoperability, and an evidence-bearing path for adding shared sources and transformations.

In programming-language theory, a grammar defines a vocabulary of symbols and rules for combining them into well-formed constructions (Aho et al., 2006). CUBEDYNAMICS uses the word more loosely and operationally. Its vocabulary consists of environmental nouns, transformations, and explicit parameters; its rules describe which objects can be combined and what each operation produces. The package does not provide a parser, generate every valid program, or claim the completeness of a formal language.

The project is organized so that different contributors can work at different layers. A scientist can use built-in data names and transformations without adopting a new data structure. A project team can add a transformation as an ordinary Python function and document its domain assumptions in the project package. A source maintainer can propose a data source together with its provider path, standardized metadata, origin and processing records, small checked example datasets, quality checks, documentation, and an executable example. Promotion into the shared vocabulary is therefore an evidentiary commitment rather than merely the addition of another function name. The complete role and contribution pathways are reported in SI Table S2. The useful unit of growth is not an isolated function, but a scientific name or operation accompanied by the information needed for others to interpret and test it.

3 Design: a readable expression and an inspectable foundation

In Python, an *expression* is a piece of syntax that can be evaluated to produce a value, whereas an assignment is a statement that binds such a value to a name (Python Software Foundation, 2026). In the example below, the code to the right of the equals sign is the analytical expression; the complete line assigns the resulting pipe object to `analysis`. A named environmental input enters the pipe and is transformed by configured operations in the order written. Beneath that expression, a compact record describes the current result, an ordered history records completed stages, and source information and evidence ground the abstraction:

```
analysis = pipe(noun) | verb(parameters) | verb(parameters)
```

The code remains Python rather than a separate formal language. The linguistic terms identify functional roles: nouns name environmental information, verbs transform it, parameters specify how an operation is performed, and authored order supplies syntax. A Python *callable* is an object that can be invoked with arguments; functions and methods are common examples (Python Software

Foundation, 2026). In this paper, *semantic* means the scientific role or interpretation of a data object; it does not imply that CUBEDYNAMICS implements a formal ontology. Table 1 defines the project’s recurring terms before they are used as shorthand.

Table 1: Operational definitions of recurring CUBEDYNAMICS terms.

Term	Meaning in this paper
Grammar	The environmental vocabulary and documented rules for combining its elements; a design structure rather than a complete formal language
Expression	The evaluable Python code on the right-hand side of the assignment, consisting here of a starting object and an ordered sequence of operations
Noun and source qualifier	A named kind of environmental information, such as temperature, together with the provider and product from which it is obtained
Verb and parameters	A named transformation and the explicit choices controlling it, such as averaging over time or applying an inclusive threshold
Callable	A Python object that can be invoked with arguments; project extensions commonly use ordinary functions as callables
Semantic contract	A documented rule stating which inputs an operation accepts and how the operation changes what the result represents
Semantic state	A runtime metadata record of the package’s declared or metadata-inferred interpretation of the result and what information, such as time variation or units, remains
Trace	The ordered record of operations, parameters, messages, and state changes completed inside the pipe
Condition	An xarray Dataset whose Boolean <code>state</code> field records where or when a stated criterion is satisfied under the package’s documented two-valued comparison rules
Observation support	The spatial area or time interval represented or summarized by a value, which is not necessarily recoverable from a coordinate label alone
Guardrail	A check that reports or rejects a known mismatch or scientifically incompatible operation rather than silently changing the data

The use of observation support follows established work on coordinate meaning, scale, aggregation, and the change-of-support problem (Hassell et al., 2017; Gotway and Young, 2002; Dungan et al., 2002).

3.1 Source-qualified nouns

A noun provides a stable scientific entry point; its required source qualifier identifies the provider and product that give it observational meaning. Ecological observation models distinguish an observed property from its procedure, feature of interest, and result (Madin et al., 2007). The W3C and OGC observation ontologies formalize related distinctions among observations, sensors, procedures, sampling, and results (Janowicz et al., 2019; Haller et al., 2019). The Climate and Forecast data model provides conventions for variables, units, coordinates, and related metadata in climate and geoscience data (Hassell et al., 2017). Thus the `prism` and `gridmet` forms of `temperature` share a vocabulary without becoming equivalent measurements. PRISM and gridMET are distinct products with different construction and intended uses (Daly et al., 2008; Abatzoglou, 2013). Their native variables, statistics, units, grids, temporal support, revisions, queries, provenance, and reviewed quality-assurance results remain source-specific within CUBEDYNAMICS (Tuff, 2026a).

This distinction makes the abstraction useful without making it careless. A common noun supports

discoverability and shared analysis code; the source qualifier prevents that convenience from becoming an implicit substitution decision. Computational compatibility after conversion or regridding is not evidence of scientific interchangeability, and data provenance remains essential to fitness for purpose (Whitfield, 2012).

3.2 Verbs, order, state, and trace

A configured verb carries the input and output rule defined in Table 1. Thresholding changes a continuous observation into a condition; for fully finite input, averaging that Boolean condition computes the unweighted arithmetic proportion of indexed positions satisfying the condition over the named dimension; event detection requires a condition with ordered time and remaining temporal variation. The present two-valued condition contract treats a missing numerical comparison as false, so this proportion is not generally the proportion among valid observations. That consequential boundary is specified in the SI and revisited in the Discussion. The semantic state must therefore describe the result produced, not simply inherit labels from the input. The maintained condition representation and the complete list of recorded state fields are documented in the SI.

The order in which verbs are authored is part of the scientific statement. CUBEDYNAMICS calls each stage once, from left to right, and does not rearrange the expression. Some orders are both valid but meaning-changing, as in Figure 1; some remove information required later; and some are conditionally equivalent only under explicit assumptions. Semantic workflow research provides close precedent for using scientific constraints to distinguish meaningful compositions from invalid ones (Gil et al., 2011; Bowers, 2012; Miksa and Rauber, 2017).

After each completed stage, semantic state records the package’s declared or metadata-inferred interpretation of the scientific role, dimensions, units, coordinate reference system, temporal and spatial status, observation support, remaining variation, source qualifier, and available source and processing history. This record does not prove that incoming metadata or scientific assumptions are correct. The trace records operations, parameters, messages, and state changes in authored order. It is a local history of work inside the pipe, not complete workflow provenance: preparation before `pipe()` and work after `unwrap()` require complementary records (Davidson and Freire, 2008; World Wide Web Consortium, 2013; Soiland-Reyes et al., 2022; Leo et al., 2024).

Alignment follows the same principle. Equal coordinate labels do not prove equal spatial footprints or temporal support. The change-of-support literature shows why differences in scale, aggregation, and support can alter analysis even when data appear computationally compatible (Gotway and Young, 2002; Dungan et al., 2002; Baumann, 2024). CUBEDYNAMICS therefore requires exact coordinates for direct composition, reports known support differences, and does not silently reproject, resample, truncate, or invent support. Researchers remain responsible for any scientifically justified harmonization.

3.3 Ordinary objects and project extensions

The adoption path is reversible: xarray object $\rightarrow$ `pipe` $\rightarrow$ inspected operations $\rightarrow$ `unwrap` $\rightarrow$ xarray object. NumPy- and Dask-backed values retain their established behavior where operations permit it. `unwrap()` returns the current value; it does not force computation, certify the result, or complete

the wider workflow.

Project extensions do not require a new data class or registration through a plugin system. An ordinary Python function that accepts the current object and returns the next object can participate in a pipe. That low barrier is important for research groups developing domain methods. Promotion into the shared package remains stricter because a public scientific name and its documented input and output rules will be reused outside the originating project. The repository therefore encourages project-owned vocabularies first, followed by broader promotion only when meaning, evidence, tests, and teaching material are ready together (Tuff, 2026a).

3.4 Evidence beneath the abstraction

Concision moves complexity; it does not eliminate it. Retrieval, variable selection, coordinate interpretation, provenance, and validation can recede behind a short noun call. Without a path downward, readable abstraction could make an analysis less inspectable. Metadata integration itself is scientific work rather than a cost-free administrative step (Edwards et al., 2011).

This motivates the design maxim:

The shorter the analytical expression becomes, the stronger the evidence underneath it must become.

A reader should be able to descend from a noun to its source, query, native variable, units, coordinates, support, revision, provenance, and quality-assurance result limited to the query and version actually tested; from a verb to its parameters, change in meaning, trace entry, and behavioral evidence. Current provider availability, the identity of a checked example dataset, metadata interpretation, numerical plausibility, and suitability for a scientific purpose answer different questions and are recorded separately. The SI provides the evidence matrix and source records.

4 Worked example: warm and dry days in South Dakota

The Working Lands vignette returns to the opening question with an explicit definition: where during July 2024 was maximum temperature at or above each cell’s empirical 0.75 quantile for that month while precipitation was at or below 0.1 mm? Both observations come from PRISM over a central South Dakota bounding box. The two conditions are constructed in separate recorded sequences and then combined:

```
from cubedynamics import data, pipe, verbs as v

query = dict(
    source="prism", bbox=[-101.2, 43.7, -100.4, 44.3],
    start="2024-07-01", end="2024-07-31",
)
temperature = data.temperature(**query, statistic="maximum")
precipitation = data.precipitation(**query)
```

```

warm_branch = pipe(temperature) | v.quantile_state(
  quantile=0.75, direction="above", name="warm_july_day"
)
dry_branch = pipe(precipitation) | v.threshold_state(
  threshold=0.1, direction="below", name="trace_or_no_rain"
)

warm = warm_branch.unwrap()
dry = dry_branch.unwrap()

analysis = (
  pipe(warm)
  | v.overlap(dry, name="warm_and_dry")
  | v.mean(dim="time", keep_dim=False)
)
frequency = analysis.unwrap()["state"]
frequency_percent = frequency * 100 # presentation outside the trace

```

The two sequences make the analytical argument visible. Temperature becomes a locally ranked warm condition; precipitation becomes a trace-or-no-rain condition; overlap produces the joint condition; and the temporal mean produces the unitless proportion of 31 indexed days satisfying both conditions. Multiplication by 100 is an explicitly untraced presentation conversion after `unwrap()`. Exact agreement of coordinates is required before overlap. Because both sequences use the same PRISM daily grid and observation support, this vignette requires no decision about reconciling different products.

Table 2: Inspectable records retained by the worked example. The entries summarize the documented state transitions; a frozen submission artifact should additionally include the exact serialized `explain()`, state, and trace output.

Retained object	Result interpretation	Record available for review
<code>warm_branch</code>	PRISM temperature condition	Source request and the <code>quantile.state</code> parameters, messages, and state transition
<code>dry_branch</code>	PRISM precipitation condition	Source request and the <code>threshold.state</code> parameters, messages, and state transition
<code>analysis</code> before unwrapping	Unitless occurrence-proportion summary	Exact-alignment requirement, overlap definition, temporal reduction, and final semantic state
Ordinary Python value	Percentage used for display	The explicit multiplication by 100 occurs after the inspected pipe and is therefore outside its trace

The reviewed example dataset contains 31 fully finite days on a 15×19 grid. The joint condition occurs on 2 to 7 days, or 6.5% to 22.6% after rounding (Tuff, 2026e). “Warm” means at or above that cell’s own quantile within this July sample; in the absence of ties, the definition selects approximately one quarter of the days at each cell by construction. Spatial variation therefore primarily reflects when the precipitation condition coincides with those locally ranked warm days. It is not a long-term climatological anomaly. Sample quantiles themselves have multiple conventional definitions, and the current implementation uses xarray’s default rather than exposing a CubeDynamics method choice (Hyndman and Fan, 1996). “Dry” is the stated precipitation threshold. The result is a co-occurrence frequency, not a diagnosis of drought, vegetation stress, land sensitivity, ecological

harm, causation, economic impact, or risk.

The example is intentionally modest. Its value is that the grammar makes it easier to say exactly what was asked, to retain the two sequence records and source identity, and to state what the result does not support. It demonstrates the combination of two source-qualified inputs, not unrestricted workflow branching or a complete history across sequences that were separately returned to ordinary Python.

5 Implementation and semantic validation

The validation asks two complementary implementation questions: do statements that meet the documented input rules execute with result descriptions that match their numerical objects, and are known incompatible statements rejected informatively? It does not test whether people understand the expressions more quickly or whether an analysis is scientifically appropriate. Scientific software often lacks a simple independently known correct answer—a test oracle—for every output, making checks of invariants, relationships, expected failures, and direct versus wrapped calculations particularly valuable (Kanewala and Bieman, 2014; Barr et al., 2015; Segura et al., 2016). A grammar should not maximize the fraction of arbitrary operation chains that run; permissiveness is a defect when a sequence contradicts its documented scientific rules.

External RC1 testing exposed both genuine defects and correct guardrails. The implementation was hardened so averaging a condition produces a summary rather than retaining an obsolete Boolean description, variance carries squared units, all condition-producing operations return the Dataset form defined in Table 1, and plotting selects the scientifically relevant field and display from the result’s meaning and dimensions. Both threshold-then-mean and mean-then-threshold now execute while retaining a note that order changes meaning. By contrast, exact-coordinate enforcement, rejection of fire and environmental data with no dates in common, and refusal to detect events after temporal variation has been removed were preserved. The SI records the complete classification of findings and the tests preventing repaired behavior from recurring (Tuff, 2026c,d).

Source and composition checks address a different layer. Noun outputs retain the source qualifier, provider and product information, native variables, queries, units, grid metadata, support, and provenance. Small fixed example datasets test a bounded retrieval and conversion path separately from whether a provider is currently reachable. The Working Lands notebook checks finite values, physical ranges, exact coordinates, agreement between direct and piped calculations, provenance, and a percentage within its expected bounds. These checks support the implemented statement and its interpretation; they do not certify every provider observation or establish fitness for a decision.

A development record accessed September 25, 2026 reports 932 passing and 5 skipped deterministic offline tests together with twelve supported notebooks using checksum-controlled observational extracts, source and vignette quality assurance, generated-reference and visual checks, strict documentation and link builds, and browser audits (Tuff, 2026d). These overlapping checks support specific documented behaviors; their counts are not an additive quality score. The five skips are not yet enumerated in the manuscript archive, and the post-rc3 record is not yet bound to an immutable submission commit. The SI therefore reports the counts as dated development evidence rather than attributing them to the public rc3 artifact.

6 Discussion

6.1 A community of environmental vocabularies

The most immediate use of CUBEDYNAMICS is not a hundred-dataset synthesis. It is helping one research team make an existing xarray analysis readable and reviewable. A scientist can wrap a familiar object, use a shared verb, inspect the current description and history, and return to ordinary Python. A domain-method developer can publish a small project vocabulary without waiting for approval from a central catalog. A source contributor can add a noun through a documented path that joins retrieval, metadata, provenance, quality assurance, tests, and an executable example. A reviewer can follow a concise expression downward rather than treating it as an opaque convenience function.

This contribution model also prevents one universal vocabulary from absorbing every scientific assumption. Core grammar behavior should remain small and general. Shared nouns and verbs should be reusable across projects. Domain assumptions belong first in project packages where their scientific community can evaluate them. The built-in fire, synchrony, terrain, road, and streamflow work demonstrates possible extension shapes; it does not make every project method part of the minimal grammar.

6.2 Toward larger environmental synthesis

Earth-system research increasingly organizes heterogeneous observations into multivariate data cubes (Mahecha et al., 2020; Montero et al., 2024). Interoperability among those systems remains an active problem (Giuliani et al., 2019), and whether data are ready for analysis depends on the task, scale, metadata, and intended interpretation (Baumann, 2024). CubeDynamics’ long-term design question is therefore not merely how to load many datasets. It is whether the analytical argument of a large synthesis can remain readable while each abstraction stays connected to source identity, support, transformations, provenance, and evidence.

The aspirational endpoint may be a page-sized specification rather than a five-line script: temperature, precipitation, vegetation, fire history, terrain, hydrology, roads, soils, species, and infrastructure entering distinct analytical paths; some summarized, some combined, and some deliberately kept separate. A scientist should be able to read that argument, a computer to execute it, and a reviewer—or software operating within explicitly limited tasks—to inspect its components. The current implementation demonstrates primarily linear pipes and a bounded combination of two inputs. It does not automatically reconcile heterogeneous products, provide a complete language for arbitrary branching, select sources autonomously, or exercise scientific judgment.

Other limitations remain. Semantic states reduce richer scientific meaning to a small recorded vocabulary and describe the package’s declared or metadata-inferred interpretation rather than verifying the truth of incoming metadata. Boolean conditions currently distinguish only true from false: missing numeric comparisons become false, so a later mean uses all indexed positions as its denominator rather than only valid observations. Analytical paths returned separately to ordinary Python do not automatically contribute their complete histories to a later trace; support metadata reveal but do not resolve product comparability; and user-supplied Python functions remain responsible for the validity of their own scientific intent. The grammar does not

represent or propagate product bias, retrieval error, calibration uncertainty, ensemble uncertainty, or covariance. Source quality-assurance records age as providers change. Complete reconstruction still requires environments, inputs, revisions, and provenance outside the pipe, consistent with established reproducible-computing guidance (Sandve et al., 2013; Wilson et al., 2017) and FAIR data principles (Wilkinson et al., 2016). Comparative studies would be required to claim improved comprehension, fewer errors, greater productivity, or better decisions.

7 Conclusion

CUBEDYNAMICS emerged from a practical need to share environmental methods without separating concise code from the observations and choices that give it meaning. Its small grammar links provider-specific environmental inputs, named transformations, order of execution, a description of the current result, a history of completed operations, ordinary xarray interoperability, and evidence limited to what was actually tested. The Working Lands vignette shows the present scale of that idea; the extension model shows how a community can add sources and domain methods without turning the core into a universal language.

The goal is not to make difficult environmental science look simple. It is to make the complexity readable, executable, extensible, and inspectable without losing the ability to verify what lies underneath.

8 Availability and release status

Source code and documentation are available at <https://github.com/CU-ESIIL/cubedynamics> and <https://cu-esiil.github.io/cubedynamics/> under the MIT License. The latest public artifact reviewed here is the 0.1.0rc3 prerelease, uploaded to PyPI on September 2, 2026 as an installable package and source archive. PyPI’s publishing record links it to source commit `f777eb07d3ad` (abbreviated) (Tuff, 2026b). It is a release candidate rather than the final 0.1.0 release. The Working Lands values were separately checked against repository commit `862a80aed8a2` (abbreviated), and the SI identifies the fixture path and checksum. Other later repository evidence remains dated but mutable and must be cited by exact commit before submission. No final release DOI has been assigned; a submission archive should bind the manuscript, evidence, and reviewed software revision in accordance with software-citation and FAIR research-software practice (Smith et al., 2016; Barker et al., 2022).

Generative AI transparency statement

The author used OpenAI Codex for LaTeX diagnostics, repository and documentation checks, structural and stylistic editing, literature review, and figure design. The author supplied and reviewed the scientific and software content; Codex did not generate observational data or assign validation outcomes. The author verified retained claims and citations and accepts responsibility for the manuscript.

Author contributions

Ty Tuff: Conceptualization, Methodology, Software, Validation, Data curation, Visualization, Writing—original draft, Writing—review and editing, and Project administration.

Competing interests

The author declares no competing interests.

Acknowledgements

The author acknowledges the maintainers of the scientific Python libraries on which CUBEDYNAMICS depends and the organizations that provide the environmental data products used in its validation examples.

References

- John T. Abatzoglou. Development of gridded surface meteorological data for ecological applications and modelling. *International Journal of Climatology*, 33(1):121–131, 2013. doi: 10.1002/joc.3413. URL <https://doi.org/10.1002/joc.3413>.
- Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2 edition, 2006. ISBN 978-0-321-48681-3. URL <https://www.informit.com/store/compilers-principles-techniques-and-tools-9780321486813>.
- Marius Appel and Edzer Pebesma. On-demand processing of data cubes from satellite image collections with the gdalcubes library. *Data*, 4(3):92, 2019. doi: 10.3390/data4030092. URL <https://doi.org/10.3390/data4030092>.
- Hannah Augustin, Martin Sudmanns, Dirk Tiede, Stefan Lang, and Andrea Baraldi. Semantic earth observation data cubes. *Data*, 4(3):102, 2019. doi: 10.3390/data4030102. URL <https://doi.org/10.3390/data4030102>.
- Michelle Barker, Neil P. Chue Hong, Daniel S. Katz, Anna-Lena Lamprecht, Carlos Martinez-Ortiz, Fotis E. Psomopoulos, Jen Harrow, Leyla Jael Castro, Morane Gruenpeter, Paula Andrea Martinez, and Tom Honeyman. Introducing the FAIR principles for research software. *Scientific Data*, 9:622, 2022. doi: 10.1038/s41597-022-01710-x. URL <https://doi.org/10.1038/s41597-022-01710-x>.
- Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, 2015. doi: 10.1109/TSE.2014.2372785. URL <https://doi.org/10.1109/TSE.2014.2372785>.
- Peter Baumann. On the analysis-readiness of spatio-temporal earth data and suggestions for its enhancement. *Environmental Modelling & Software*, 176:106017, 2024. doi: 10.1016/j.envsoft.2024.106017. URL <https://doi.org/10.1016/j.envsoft.2024.106017>.

- Pascal Bourgault, David Huard, Trevor James Smith, Travis Logan, Abel Aoun, Juliette Lavoie, et al. xclim: xarray-based climate data analytics. *Journal of Open Source Software*, 8(85):5415, 2023. doi: 10.21105/joss.05415. URL <https://doi.org/10.21105/joss.05415>.
- Shawn Bowers. Scientific workflow, provenance, and data modeling challenges and approaches. *Journal on Data Semantics*, 1:19–30, 2012. doi: 10.1007/s13740-012-0004-y. URL <https://doi.org/10.1007/s13740-012-0004-y>.
- Ed H. Chi. A taxonomy of visualization techniques using the data state reference model. In *Proceedings of the IEEE Symposium on Information Visualization 2000*, pages 69–75, 2000. doi: 10.1109/INFVIS.2000.885092. URL <https://doi.org/10.1109/INFVIS.2000.885092>.
- clisops contributors. clisops: Climate simulation operations. Computer software and documentation, 2026. URL <https://clisops.readthedocs.io/>.
- Michael R. Crusoe, Sanne Abeln, Alexandru Iosup, Peter Amstutz, John Chilton, Nebojša Tijanić, Hervé Ménager, Stian Soiland-Reyes, Bogdan Gavrilović, Carole Goble, and CWL Community. Methods included: Standardizing computational reuse and portability with the common workflow language. *Communications of the ACM*, 65(6):54–63, 2022. doi: 10.1145/3486897. URL <https://doi.org/10.1145/3486897>.
- Christopher Daly, Michael Halbleib, Joseph I. Smith, Wayne P. Gibson, Matthew K. Doggett, George H. Taylor, Jan Curtis, and Phillip P. Pasteris. Physiographically sensitive mapping of climatological temperature and precipitation across the conterminous United States. *International Journal of Climatology*, 28(15):2031–2064, 2008. doi: 10.1002/joc.1688. URL <https://doi.org/10.1002/joc.1688>.
- Susan B. Davidson and Juliana Freire. Provenance and scientific workflows: Challenges and opportunities. In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, pages 1345–1350, 2008. doi: 10.1145/1376616.1376772. URL <https://doi.org/10.1145/1376616.1376772>.
- Ewa Deelman, Karan Vahi, Gideon Juve, Mats Rynge, Scott Callaghan, Philip J. Maechling, Rajiv Mayani, Weiwei Chen, Rafael Ferreira da Silva, Miron Livny, and Kent Wenger. Pegasus, a workflow management system for science automation. *Future Generation Computer Systems*, 46:17–35, 2015. doi: 10.1016/j.future.2014.10.008. URL <https://doi.org/10.1016/j.future.2014.10.008>.
- Jennifer L. Dungan, Joe N. Perry, Mark R. T. Dale, Pierre Legendre, Sandra Citron-Pousty, Marie-Josée Fortin, Agnieszka Jakomulska, Maria N. Miriti, and Michael S. Rosenberg. A balanced view of scaling in spatial statistical analysis. *Ecography*, 25(5):626–640, 2002. doi: 10.1034/j.1600-0587.2002.250510.x. URL <https://doi.org/10.1034/j.1600-0587.2002.250510.x>.
- John L. Dwyer, David P. Roy, Brian Sauer, Calli B. Jenkerson, Hankui K. Zhang, and Leo Lymburner. Analysis ready data: Enabling analysis of the landsat archive. *Remote Sensing*, 10(9):1363, 2018. doi: 10.3390/rs10091363. URL <https://doi.org/10.3390/rs10091363>.
- Paul N. Edwards, Matthew S. Mayernik, Archer L. Batcheller, Geoffrey C. Bowker, and Christine L. Borgman. Science friction: Data, metadata, and collaboration. *Social Studies of Science*, 41(5):667–690, 2011. doi: 10.1177/0306312711413314. URL <https://doi.org/10.1177/0306312711413314>.

- Esri. What is ModelBuilder? ArcGIS Pro documentation, 2026. URL <https://pro.arcgis.com/en/pro-app/latest/help/analysis/geoprocessing/modelbuilder/what-is-modelbuilder-.htm>. Accessed August 28, 2026.
- Bakinam T. Essawy, Jonathan L. Goodall, Daniel Voce, Mohamed M. Morsy, Jeffrey M. Sadler, Young Don Choi, David G. Tarboton, and Tanu Malik. A taxonomy for reproducible and replicable research in environmental modelling. *Environmental Modelling & Software*, 134:104753, 2020. doi: 10.1016/j.envsoft.2020.104753. URL <https://doi.org/10.1016/j.envsoft.2020.104753>.
- Ben Fry. *Visualizing Data: Exploring and Explaining Data with the Processing Environment*. O’Reilly Media, 2007. ISBN 978-0-596-51455-6. URL <https://www.oreilly.com/library/view/visualizing-data/9780596514556/>.
- Benjamin Jotham Fry. *Computational Information Design*. PhD thesis, Massachusetts Institute of Technology, 2004. URL <https://www.benfry.com/phd/>.
- Yolanda Gil, Varun Ratnakar, Jihie Kim, Pedro Antonio Gonzalez-Calero, Paul Groth, Joshua Moody, and Ewa Deelman. Wings: Intelligent workflow-based design of computational experiments. *IEEE Intelligent Systems*, 26(1):62–72, 2011. doi: 10.1109/MIS.2010.9. URL <https://doi.org/10.1109/MIS.2010.9>.
- Sean Gillies and contributors. Rasterio: Geospatial raster I/O for Python programmers. Computer software, 2013–present. URL <https://github.com/rasterio/rasterio>.
- Sean Gillies, Casper van der Wel, Joris Van den Bossche, et al. Shapely, version 2.1.1. Computer software, 2025. URL <https://doi.org/10.5281/zenodo.5597138>.
- Gregory Giuliani, Bruno Chatenoux, Andrea De Bono, Denisa Rodila, Jean-Philippe Richard, Karin Allenbach, Hy Dao, and Pascal Peduzzi. Building an Earth Observations Data Cube: lessons learned from the Swiss Data Cube (SDC) on generating Analysis Ready Data (ARD). *Big Earth Data*, 1(1–2):100–117, 2017. doi: 10.1080/20964471.2017.1398903. URL <https://doi.org/10.1080/20964471.2017.1398903>.
- Gregory Giuliani, Joan Masó, Paolo Mazzetti, Stefano Nativi, and Alaitz Zabala. Paving the way to increased interoperability of earth observations data cubes. *Data*, 4(3):113, 2019. doi: 10.3390/data4030113. URL <https://doi.org/10.3390/data4030113>.
- Steven N. Goodman, Daniele Fanelli, and John P. A. Ioannidis. What does research reproducibility mean? *Science Translational Medicine*, 8(341):341ps12, 2016. doi: 10.1126/scitranslmed.aaf5027. URL <https://doi.org/10.1126/scitranslmed.aaf5027>.
- Carol A. Gotway and Linda J. Young. Combining incompatible spatial data. *Journal of the American Statistical Association*, 97(458):632–648, 2002. doi: 10.1198/016214502760047140. URL <https://doi.org/10.1198/016214502760047140>.
- Armin Haller, Krzysztof Janowicz, Simon J. D. Cox, Maxime Lefrançois, Kerry Taylor, Danh Le Phuoc, Joshua Lieberman, Raúl García-Castro, Rob Atkinson, and Claus Stadler. The modular SSN ontology: A joint W3C and OGC standard specifying the semantics of sensors, observations, sampling, and actuation. *Semantic Web*, 10(1):9–32, 2019. doi: 10.3233/SW-180320. URL <https://doi.org/10.3233/SW-180320>.

- Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, et al. Array programming with NumPy. *Nature*, 585:357–362, 2020. doi: 10.1038/s41586-020-2649-2. URL <https://doi.org/10.1038/s41586-020-2649-2>.
- David Hassell, Jonathan Gregory, Jon Blower, Bryan N. Lawrence, and Karl E. Taylor. A data model of the climate and forecast metadata conventions (CF-1.6) with a software implementation (cf-python v2.1). *Geoscientific Model Development*, 10:4619–4646, 2017. doi: 10.5194/gmd-10-4619-2017. URL <https://doi.org/10.5194/gmd-10-4619-2017>.
- Stephan Hoyer and Joe Hamman. xarray: N-D labeled arrays and datasets in Python. *Journal of Open Research Software*, 5(1):10, 2017. doi: 10.5334/jors.148. URL <https://doi.org/10.5334/jors.148>.
- Rob J. Hyndman and Yanan Fan. Sample quantiles in statistical packages. *The American Statistician*, 50(4):361–365, 1996. doi: 10.1080/00031305.1996.10473566. URL <https://doi.org/10.1080/00031305.1996.10473566>.
- intake-esm contributors. intake-esm: Cataloging and loading earth system model data. Computer software, 2026. URL <https://github.com/intake/intake-esm>.
- Krzysztof Janowicz, Armin Haller, Simon J. D. Cox, Danh Le Phuoc, and Maxime Lefrançois. SOSA: A lightweight ontology for sensors, observations, samples, and actuators. *Journal of Web Semantics*, 56:1–10, 2019. doi: 10.1016/j.websem.2018.06.003. URL <https://doi.org/10.1016/j.websem.2018.06.003>.
- Kelsey Jordahl, Joris Van den Bossche, Martin Fleischmann, Jacob Wasserman, James McBride, Jeffrey Gerard, et al. geopandas/geopandas, version 0.8.1. Computer software, 2020. URL <https://doi.org/10.5281/zenodo.3946761>.
- Upulee Kanewala and James M. Bieman. Testing scientific software: A systematic literature review. *Information and Software Technology*, 56(10):1219–1232, 2014. doi: 10.1016/j.infsof.2014.05.006. URL <https://doi.org/10.1016/j.infsof.2014.05.006>.
- Donald E. Knuth. Literate programming. *The Computer Journal*, 27(2):97–111, 1984. doi: 10.1093/comjnl/27.2.97. URL <https://doi.org/10.1093/comjnl/27.2.97>.
- Simone Leo, Michael R. Crusoe, Laura Rodríguez-Navas, Raúl Sirvent, Alexander Kanitz, Paul De Geest, Rudolf Wittner, Luca Pireddu, Daniel Garijo, José M. Fernández, et al. Recording provenance of workflow runs with RO-Crate. *PLOS ONE*, 19(9):e0309210, 2024. doi: 10.1371/journal.pone.0309210. URL <https://doi.org/10.1371/journal.pone.0309210>.
- Meng Lu, Marius Appel, and Edzer Pebesma. Multidimensional arrays for analysing geoscientific data. *ISPRS International Journal of Geo-Information*, 7(8):313, 2018. doi: 10.3390/ijgi7080313. URL <https://doi.org/10.3390/ijgi7080313>.
- Joshua Madin, Shawn Bowers, Mark Schildhauer, Sergeui Krivov, Deana Pennington, and Ferdinando Villa. An ontology for describing and synthesizing ecological observation data. *Ecological Informatics*, 2(3):279–296, 2007. doi: 10.1016/j.ecoinf.2007.05.004. URL <https://doi.org/10.1016/j.ecoinf.2007.05.004>.
- Miguel D. Mahecha, Fabian Gans, Gunnar Brandt, Rune Christiansen, Sarah E. Cornell, Normann Fomferra, Guido Kraemer, Jonas Peters, Paul Bodesheim, Gustau Camps-Valls, Jonathan F.

- Donges, Wouter Dorigo, Lina M. Estupinan-Suarez, Victor H. Gutierrez-Velez, Martin Gutwin, Martin Jung, Maria C. Londoño, Diego G. Miralles, Phillip Papastefanou, and Markus Reichstein. Earth system data cubes unravel global multivariate dynamics. *Earth System Dynamics*, 11:201–234, 2020. doi: 10.5194/esd-11-201-2020. URL <https://doi.org/10.5194/esd-11-201-2020>.
- Tomasz Miksa and Andreas Rauber. Using ontologies for verification and validation of workflow-based experiments. *Journal of Web Semantics*, 43:25–45, 2017. doi: 10.1016/j.websem.2017.01.002. URL <https://doi.org/10.1016/j.websem.2017.01.002>.
- David Montero, Guido Kraemer, Anca Angheloa, César Aybar, Gunnar Brandt, Gustau Camps-Valls, Felix Cremer, Ida Flik, Fabian Gans, Sarah Habershon, Chaonan Ji, Teja Kattenborn, Laura Martínez-Ferrer, Francesco Martinuzzi, Martin Reinhardt, Maximilian Söchting, Khalil Teber, and Miguel D. Mahecha. Earth system data cubes: Avenues for advancing earth system research. *Environmental Data Science*, 3:e27, 2024. doi: 10.1017/eds.2024.22. URL <https://doi.org/10.1017/eds.2024.22>.
- National Academies of Sciences, Engineering, and Medicine. *Reproducibility and Replicability in Science*. National Academies Press, Washington, DC, 2019. doi: 10.17226/25303. URL <https://doi.org/10.17226/25303>.
- Roger D. Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011. doi: 10.1126/science.1213847. URL <https://doi.org/10.1126/science.1213847>.
- Python Software Foundation. Python 3.14 glossary. Python language documentation, 2026. URL <https://docs.python.org/3/glossary.html>. Definitions of expression and callable; accessed September 25, 2026.
- Mattia Righi, Bouwe Andela, Veronika Eyring, Axel Lauer, Valeriu Predoi, Manuel Schlund, et al. Earth system model evaluation tool (ESMValTool) v2.0—technical overview. *Geoscientific Model Development*, 13:1179–1199, 2020. doi: 10.5194/gmd-13-1179-2020. URL <https://doi.org/10.5194/gmd-13-1179-2020>.
- Matthew Rocklin. Dask: Parallel computation with blocked algorithms and task scheduling. In *Proceedings of the 14th Python in Science Conference*, pages 126–132, 2015. doi: 10.25080/Majora-7b98e3ed-013. URL <https://doi.org/10.25080/Majora-7b98e3ed-013>.
- Adam Rule, Aurélien Tabard, and James D. Hollan. Exploration and explanation in computational notebooks. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, pages 1–12, 2018. doi: 10.1145/3173574.3173606. URL <https://doi.org/10.1145/3173574.3173606>.
- Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. Ten simple rules for reproducible computational research. *PLoS Computational Biology*, 9(10):e1003285, 2013. doi: 10.1371/journal.pcbi.1003285. URL <https://doi.org/10.1371/journal.pcbi.1003285>.
- Michael Schramm, Edzer Pebesma, Milutin Milenković, Luca Foresta, Jeroen Dries, Alexander Jacob, Wolfgang Wagner, Matthias Mohr, Markus Neteler, Matej Kadunc, Tomasz Miksa, Pieter Kempeneers, Jan Verbesselt, Bernd Gößwein, Christian Navacchi, Sander Lippens, and Johannes Reiche. The openEO API—harmonising the use of earth observation cloud services using virtual data cube functionalities. *Remote Sensing*, 13(6):1125, 2021. doi: 10.3390/rs13061125. URL <https://doi.org/10.3390/rs13061125>.

- Sergio Segura, Gordon Fraser, Ana B. Sánchez, and Antonio Ruiz-Cortés. A survey on metamorphic testing. *IEEE Transactions on Software Engineering*, 42(9):805–824, 2016. doi: 10.1109/TSE.2016.2532875. URL <https://doi.org/10.1109/TSE.2016.2532875>.
- Arfon M. Smith, Daniel S. Katz, Kyle E. Niemeyer, and FORCE11 Software Citation Working Group. Software citation principles. *PeerJ Computer Science*, 2:e86, 2016. doi: 10.7717/peerj-cs.86. URL <https://doi.org/10.7717/peerj-cs.86>.
- Alan D. Snow, R. Braun, RichardScottOZ, Martin Raspaud, David Brochart, Kirill Kouzoubov, et al. rioxarray, version 0.22.0. Computer software, 2026. URL <https://doi.org/10.5281/zenodo.18892731>.
- Stian Soiland-Reyes, Peter Sefton, Mercè Crosas, Leyla Jael Castro, Frederik Coppens, José M. Fernández, Daniel Garijo, Björn Grüning, Marco La Rosa, Simone Leo, Eoghan Ó Carragáin, Marc Portier, Ana Trisovic, Paul Groth, Carole Goble, and RO-Crate Community. Packaging research artefacts with RO-Crate. *Data Science*, 5(2):97–138, 2022. doi: 10.3233/DS-210053. URL <https://doi.org/10.3233/DS-210053>.
- SpatioTemporal Asset Catalog community. SpatioTemporal Asset Catalog specification, version 1.1.0. Software specification, 2024. URL <https://github.com/radianteearth/stac-spec/releases/tag/v1.1.0>.
- Sara Stoudt, Valéri N. Vásquez, and Ciera C. Martinez. Principles for data analysis workflows. *PLoS Computational Biology*, 17(3):e1008770, 2021. doi: 10.1371/journal.pcbi.1008770. URL <https://doi.org/10.1371/journal.pcbi.1008770>.
- Ty Tuff. CubeDynamics: A composable grammar for spatiotemporal science. Computer software and accompanying documentation, 2026a. URL <https://github.com/CU-ESIIL/cubedynamics>. Development repository and documentation; accessed September 25, 2026.
- Ty Tuff. CubeDynamics 0.1.0rc3. Public release candidate on PyPI, 2026b. URL <https://pypi.org/project/cubedynamics/0.1.0rc3/>. Released September 2, 2026; wheel and source distribution published through Trusted Publishing; attested source commit f777eb07d3ada8bd407b027f560727c90f6d3731.
- Ty Tuff. CubeDynamics rc1 comprehensive validation triage. Project validation report, 2026c. URL https://github.com/CU-ESIIL/cubedynamics/blob/main/docs/project/rc1_validation_triage.md. Accessed September 25, 2026.
- Ty Tuff. CubeDynamics prompt log: semantic, release, and validation record. Project validation record, 2026d. URL https://github.com/CU-ESIIL/cubedynamics/blob/main/PROMPT_LOG.md. Accessed September 25, 2026.
- Ty Tuff. Working lands: Read hot-and-dry weather as two nouns. Executable CubeDynamics worked environmental vignette, 2026e. URL https://github.com/CU-ESIIL/cubedynamics/blob/862a80aed8a2781b40e6e5293fd6cfbcba887aa4/docs/decision_vignettes/working_lands.ipynb. Reviewed at source commit 862a80aed8a2781b40e6e5293fd6cfbcba887aa4; accessed September 25, 2026.
- Paul H. Whitfield. Why the provenance of data matters: Assessing fitness for purpose for environmental data. *Canadian Water Resources Journal*, 37(1):23–36, 2012. doi: 10.4296/cwrj3701866. URL <https://doi.org/10.4296/cwrj3701866>.

- Hadley Wickham. A layered grammar of graphics. *Journal of Computational and Graphical Statistics*, 19(1):3–28, 2010. doi: 10.1198/jcgs.2009.07098. URL <https://doi.org/10.1198/jcgs.2009.07098>.
- Hadley Wickham. Tidy data. *Journal of Statistical Software*, 59(10):1–23, 2014. doi: 10.18637/jss.v059.i10. URL <https://doi.org/10.18637/jss.v059.i10>.
- Hadley Wickham, Mara Averick, Jennifer Bryan, Winston Chang, Lucy D’Agostino McGowan, Romain François, et al. Welcome to the Tidyverse. *Journal of Open Source Software*, 4(43):1686, 2019. doi: 10.21105/joss.01686. URL <https://doi.org/10.21105/joss.01686>.
- Leland Wilkinson. *The Grammar of Graphics*. Springer, 2 edition, 2005. doi: 10.1007/0-387-28695-0. URL <https://doi.org/10.1007/0-387-28695-0>.
- Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, et al. The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3:160018, 2016. doi: 10.1038/sdata.2016.18. URL <https://doi.org/10.1038/sdata.2016.18>.
- Greg Wilson, Jennifer Bryan, Karen Cranston, Justin Kitzes, Lex Nederbragt, and Tracy K. Teal. Good enough practices in scientific computing. *PLoS Computational Biology*, 13(6):e1005510, 2017. doi: 10.1371/journal.pcbi.1005510. URL <https://doi.org/10.1371/journal.pcbi.1005510>.
- World Wide Web Consortium. PROV-O: The PROV ontology. W3C Recommendation, 2013. URL <https://www.w3.org/TR/prov-o/>.
- Mingda Zhang, Xiaoqian Bu, and Peng Yue. GeoJModelBuilder: an open source geoprocessing workflow tool. *Open Geospatial Data, Software and Standards*, 2:8, 2017. doi: 10.1186/s40965-017-0022-7. URL <https://doi.org/10.1186/s40965-017-0022-7>.